

Full handbook

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

- Track A: Databricks pipeline engineering.....2**
 - A3 Delta Lake 3
 - A3-L1 Idempotent MERGE, and why you deduplicate the source first..... 3
 - Track A best-practice checklist 10
- Track B: Python for data science11**
 - B3 Clean pandas 12
 - B3-L2 Vectorise instead of apply 12
 - Track B best-practice checklist 17
- Track C: Microsoft Fabric18**
 - C2 Lakehouse, Warehouse and SQL analytics endpoint 19
 - C2-L3 Lakehouse, Warehouse or SQL analytics endpoint..... 19
 - Track C best-practice checklist 25
- Track D: Finding errors in code26**
 - D3 Silent errors 27
 - D3-L3 Mutable defaults and the truthiness of zero 27
 - Track D best-practice checklist..... 32
- Appendix: solutions and answer keys33**

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A3 Delta Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. MERGE, idempotent writes, schema enforcement and evolution, time travel, maintenance.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

Concept

A pipeline is **idempotent** when running it twice with the same input leaves the table in the same state as running it once. Retries, reruns and redeliveries happen every week in production, so idempotence is not optional.

An append is not idempotent. Run `write.mode("append")` twice and every row lands twice. A Delta `MERGE` is idempotent when two conditions hold:

1. The `ON` condition covers every column of the table's grain. For a silver table with one row per respondent per wave, that means `respondent_id` **and** `wave_id`.
2. The source holds at most one row per key.

The second condition catches people out. When two source rows match the same target row, Delta cannot tell which one should win. It stops the `MERGE` with the error `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. When neither row matches a target row, both rows are inserted, and your grain is broken without any error.

So deduplicate the source before you merge. Rank the rows for each key by a version column, such as `delivery_version` or `ingestion_timestamp`, and keep the newest. Use a window with `row_number()`. Do not use `dropDuplicates()`: it keeps an arbitrary row per key, so a rerun keeps a different version with no warning.

The pattern for every silver load:

- Keep the newest source row per key.
- `MERGE` on the full grain.
- `whenMatchedUpdateAll()` for redelivered rows, `whenNotMatchedInsertAll()` for new ones.

Bronze stays append-only, because bronze records every delivery as received. Silver is where you resolve versions.

Why it matters

A supplier redelivers a wave with corrected weights. The file repeats 37 respondents, because the supplier's export joined two tables. The silver job uses the reference `upsert_silver`, which merges without deduplicating. The `MERGE` fails at 02:00 with

`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Gold does not refresh, and the morning Power BI report shows last week's numbers.

The on-call engineer "fixes" it by switching the job to append. The next run succeeds. Silver now holds the wave twice. Reach percentages look normal, because both the numerator and the denominator doubled. Weighted population estimates are twice their true size for a month, until a reconciliation check against the weight total catches it.

Deduplicating on the full grain before the `MERGE` prevents both failures.

Worked example: weak code

```
1 from delta.tables import DeltaTable
2 from pyspark.sql import DataFrame, SparkSession
3
4 spark = SparkSession.builder.getOrCreate()
5
6
7 def upsert_silver(new_df: DataFrame, target_table: str, merge_keys: list)
  -> None:
8     """Merge new data into silver table."""
9     delta_table = DeltaTable.forName(spark, target_table)
10    merge_condition = " AND ".join(
11        [f"target.{k} = source.{k}" for k in merge_keys]
12    )
13    (
14        delta_table.alias("target")
15        .merge(new_df.alias("source"), merge_condition)
16        .whenMatchedUpdateAll()
17        .whenNotMatchedInsertAll()
18        .execute()
19    )
```

1. **Line 4.** A session created at import time. Tests cannot inject a local session, and importing the module has a side effect.
2. **Line 7.** `list` with no element type. mypy cannot check that callers pass column names.

3. **Line 15.** The source goes straight into the MERGE. One repeated key in a redelivery fails the whole job.

Worked example: strong code

```
1 from delta.tables import DeltaTable
2 from pyspark.sql import DataFrame, SparkSession, Window
3 from pyspark.sql import functions as F
4
5
6 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
    DataFrame:
7     """Keep the newest row for each key.
8
9     Args:
10         df: Source rows, possibly with repeated keys.
11         keys: Columns that identify one target row.
12         order_col: Column that ranks versions. Highest wins.
13
14     Returns:
15         One row per key.
16     """
17
18     window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
19     return (
20         df.withColumn("_rank", F.row_number().over(window))
21         .filter(F.col("_rank") == 1)
22         .drop("_rank")
23     )
24
25 def upsert_silver(
26     spark: SparkSession,
27     source: DataFrame,
28     target_table: str,
29     keys: list[str],
30     order_col: str,
31 ) -> None:
32     """MERGE the newest source row per key into a silver table.
33
34     Running it twice with the same source leaves the table unchanged.
35     """
36     condition = " AND ".join(f"t.{k} = s.{k}" for k in keys)
37     (
38         DeltaTable.forName(spark, target_table).alias("t")
39         .merge(latest_per_key(source, keys, order_col).alias("s"),
40             condition)
41         .whenMatchedUpdateAll()
42         .whenNotMatchedInsertAll()
43         .execute()
```

1. **Line 3.** Import the module as `F`. Importing `sum` or `max` by name shadows the Python built-ins.
2. **Line 17.** One partition per key, newest version first. The result is the same on every run.
3. **Line 19.** `row_number()` gives exactly one row rank 1 per key, even when versions tie. `rank()` would keep ties.
4. **Line 26.** The session is a parameter. A test passes a local session, a job passes the cluster session.
5. **Line 39.** Deduplicate inside the MERGE call, so no caller can skip it.

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

Recap card

- **When is a pipeline idempotent?** When running it twice with the same input leaves the table as one run would.
- **Two conditions for an idempotent Delta MERGE** The ON condition covers the full grain, and the source has at most one row per key.
- **What happens when two source rows match one target row?** The MERGE fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`.
- **Why not dropDuplicates() before a MERGE?** It keeps an arbitrary row per key. Use `row_number()` over a window ordered by version, newest first.

Practice

Exercise 1. Multiple choice (level 1)

The silver table holds one row for respondent `R1` in wave `W1`. A redelivery holds **two** rows for `R1` in `W1`. You run a MERGE on `respondent_id` and `wave_id` with `whenMatchedUpdateAll()` and `whenNotMatchedInsertAll()`. What happens?

- a) The target row takes the values of the last source row.
- b) The MERGE fails with an error.
- c) Both source rows are inserted as new rows.
- d) The target row takes the values of the first source row.

Why?

- r1) Two source rows match one target row, and Delta cannot tell which should win.
- r2) Delta enforces a primary key on the target table.
- r3) The source DataFrame must be cached before a MERGE.

Exercise 2. Fill the gap (level 2)

Complete the chain so that redelivered rows update, new rows insert, and the operation runs.

```
1 (
2     DeltaTable.forName(spark,
3         "research_silver.nat_respondents").alias("t")
4     .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
5         t.wave_id = s.wave_id")
6     ._____ (1) ()
7     ._____ (2) ()
8     ._____ (3) ()
9 )
```

Exercise 3. Predict the output (level 2)

`silver` starts empty. `source` holds 3 rows with 3 distinct (`respondent_id`, `wave_id`) keys. `load()` calls the strong `upsert_silver` from the worked example. What does the last line print?

```
1 load(source) # MERGE on respondent_id and wave_id
2 load(source) # the scheduler retries the same run
3 print(spark.table("silver").count())
```

Exercise 4. Spot the bug (level 3)

Silver has one row per respondent per wave. After wave `w2` loads, every wave `w1` row has disappeared. Tap the line with the bug, then fix the code.

```
1 def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2     """MERGE one wave of respondents into silver. Grain: respondent per
3     wave."""
4     latest = latest_per_key(source, ["respondent_id", "wave_id"],
5         "delivery_version")
6     (
7         DeltaTable.forName(spark,
8             "research_silver.nat_respondents").alias("t")
9         .merge(latest.alias("s"), "t.respondent_id = s.respondent_id")
10        .whenMatchedUpdateAll()
11        .whenNotMatchedInsertAll()
12        .execute()
13    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `latest_per_key(df, keys, order_col)`. It returns one row per key: the row with the highest `order_col`. Use type hints and a docstring. The checker reads your code without running Spark.

Starter code:

```
1 from pyspark.sql import DataFrame, Window
2 from pyspark.sql import functions as F
3
4
5 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
    DataFrame:
6     ...
```

Exercise 6. Put in order (level 5)

Put the steps of an idempotent silver load in order.

- Write one record to the pipeline run log
- Rename and cast columns from the Variable Registry
- Validate the bronze delivery against its data contract
- MERGE into silver on `respondent_id` and `wave_id`
- Run the table health check on silver
- Keep the newest row per respondent and wave

Practical: Prove a MERGE is idempotent

Platform: Databricks Free Edition. Run this in Databricks Free Edition on serverless compute. It creates a schema called `learning_a3` in your current catalog, loads the same data twice, and shows that the row count does not change. It then repeats a key and shows the error.

Download the notebook `A3-L1.ipynb` from the app.

Sources

- MERGE fails with `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE` when several source rows match one target row. (verified: `practice/tests/test_a3_l1_merge.py` reproduces the error on delta-spark 4.0.1. Docs: <https://docs.databricks.com/aws/en/sql/language-manual/delta-merge-into>)
- `DeltaTable.forName`, `alias`, `merge`, `whenMatchedUpdateAll`, `whenNotMatchedInsertAll`, `whenNotMatchedBySourceDelete` and `execute`

exist in delta-spark 4.0. (verified: `practice/tests/test_a3_l1_merge.py` checks the methods on delta-spark 4.0.1.)

- Free Edition users can create schemas in the workspace catalog, whose name matches the workspace. (search-only: <https://docs.databricks.com/aws/en/catalogs/default>)

Track A best-practice checklist

- ☐ Every function has type hints, a docstring and one job.
- ☐ No SparkSession is created at import time. Functions receive spark as a parameter.
- ☐ Functions are imported as F: `from pyspark.sql import functions as F`.
- ☐ Table names, column maps and thresholds come from config or registry tables.
- ☐ Every table states its grain in a comment or table property.
- ☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.
- ☐ Every run writes one record to the run log table with an explicit schema.
- ☐ CRITICAL checks raise. WARN checks log and continue.
- ☐ Every division has a guard for zero and null.
- ☐ Counts come from one agg pass, not repeated count() calls.
- ☐ Incremental loads compare source and target on the same keys.
- ☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.
- ☐ Tests run on a local SparkSession in CI before every merge.

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B3 Clean pandas

Roadmap: Prerequisite for all months. Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L2 Vectorise instead of apply

Concept

pandas stores each column as an array. A **vectorised** operation, such as `df["a"] * df["b"]`, `np.where()` or `pd.cut()`, runs one compiled loop over the whole array. `df.apply(func, axis=1)` does the opposite. It builds a Series for each row and calls your Python function once per row.

The gap is large. On one million rows, a row-wise `apply` that picks a weight when a flag is set took 4.0 seconds. `np.where` did the same job in 7.5 milliseconds, about 500 times faster. (pandas 3.0.6, NumPy 2.4.6, one laptop core.)

Speed is not the only gain. Vectorised code states the rule for the whole column in one line, so a reviewer checks one expression instead of a branching function.

Reach for these first:

Task	Vectorised tool
Arithmetic between columns	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> on Series
Two-way choice	<code>np.where(cond, a, b)</code>
Many-way choice	<code>np.select(conditions, choices, default)</code>
Numeric bands	<code>pd.cut(values, bins, labels, right=False)</code>
Lookup from a dict	<code>Series.map(mapping)</code>
Limits	<code>Series.clip(lower, upper)</code>

Watch missing values. `NaN > 60` is `False`, so `np.where(minutes > 60, "heavy", "light")` labels a missing value "light". Handle missing values on purpose: `pd.cut` keeps `NaN` as missing, and `Series.where(values.notna())` puts it back after `np.where`.

Keep `apply` for work with no vectorised form, such as calling an external parser on each string. Even then, try `Series.map` or a list comprehension on one column before `apply(axis=1)`.

Why it matters

A weighting script derives 25 columns with `apply(axis=1)`. It ran in 8 minutes on 2 million rows, so nobody noticed. The programme then merged three trackers into one file of 40 million rows. At the measured rate of about 4 seconds per million rows per column, the derivations alone need roughly 67 minutes. The job hit its 60-minute timeout every night for a week, and weighted results for all three trackers were late.

The same script also labelled respondents with a missing `minutes` value as "light" viewers, because the row function used `if row["minutes"] > 60`. The vectorised rewrite fixed the speed, and review of the one-line rule exposed the missing-value bug.

Worked example: weak code

```
1 import pandas as pd
2
3
4 def derive(df):
5     def per_row(row):
6         weighted_reach = row["final_weight"] if row["reach_flag"] == 1
7         else 0.0
8         if row["age"] < 35:
9             band = "16-34"
10        elif row["age"] < 55:
11            band = "35-54"
12        else:
13            band = "55+"
14        return pd.Series({"weighted_reach": weighted_reach, "age_group":
15            band})
16    df[["weighted_reach", "age_group"]] = df.apply(per_row, axis=1)
17    return df
```

1. **Line 4.** No type hints and no docstring. The reader cannot tell what the frame must contain.
2. **Line 7.** `NaN < 35` is False, so a missing age falls through to `55+` without a warning.
3. **Line 15.** Builds a Series and calls Python for every row. About 500 times slower than a vectorised rule.
4. **Line 15.** Mutates the caller's frame. A pure function returns a new frame.

Worked example: strong code

```
1 import numpy as np
2 import pandas as pd
3
4 AGE_BINS = [16, 35, 55, np.inf]
```

```

5 AGE_LABELS = ["16-34", "35-54", "55+"]
6
7
8 def derive(df: pd.DataFrame) -> pd.DataFrame:
9     """Add weighted reach and age group.
10
11     Args:
12         df: Respondents with reach_flag, final_weight and age.
13
14     Returns:
15         A new frame. Missing ages give a missing age_group.
16     """
17     return df.assign(
18         weighted_reach=np.where(df["reach_flag"] == 1,
19                                df["final_weight"], 0.0),
20         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
21                          right=False),
22     )

```

1. **Line 4.** Band edges live in one constant, so a change to the bands is a one-line change.
2. **Line 17.** `assign` returns a new frame, so the caller's data is untouched.
3. **Line 18.** One compiled pass over the column.
4. **Line 19.** `right=False` makes each band include its lower edge: 35 goes to 35-54. Missing ages stay missing.

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Recap card

- **Why is `df.apply(func, axis=1)` slow?** It calls a Python function once per row. `np.where` measured about 500 times faster on one million rows.
- **Vectorised if-else in NumPy** `np.where(cond, value_if_true, value_if_false)`. For several conditions, `np.select`.
- **What does `NaN > 60` return?** `False`. `np.where` and `np.select` send missing values to a real category unless you handle them.
- **Numeric bands that keep missing values missing** `pd.cut(values, bins, labels, right=False)`.

Practice

Exercise 1. Multiple choice (level 1)

Which line computes `weighted_reach` for a frame with one million rows in a single compiled pass?

- a) `df.apply(lambda r: r["final_weight"] if r["reach_flag"] == 1 else 0.0, axis=1)`
- b) `[w if f == 1 else 0.0 for w, f in zip(df["final_weight"], df["reach_flag"])]`
- c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
- d) `for i, row in df.iterrows(): ...`

Why?

- r1) It evaluates the condition and the choice on whole arrays in compiled code.
- r2) It uses fewer characters than the other options.
- r3) NumPy runs the lambda on several cores.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import pandas as pd
2
3 s = pd.Series([0.0, 1.5, None])
4 print((s > 1).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete the line so rows with `reach_flag == 1` keep their weight and every other row gets zero.

```
1 df["weighted_reach"] = np.__(1)(df["reach_flag"] == 1,
    df["final_weight"], ____(2))
```

Exercise 4. Write the function (level 3)

Write `weighted_reach_pct(df)`. It returns the weighted share of rows with `reach_flag == 1`: the sum of `final_weight` where the flag is 1, divided by the sum of all `final_weight`. Return `None` when the total weight is zero or the frame is empty. Do not use `apply` or a loop over rows.

Starter code:

```
1 import numpy as np
2 import pandas as pd
3
```

```

4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     ...

```

Exercise 5. Refactor (level 4)

Refactor `add_age_group` to vectorised code. Keep the bands: under 35 is 16-34, under 55 is 35-54, anything else is 55+. A missing age must give a missing age group, not 55+. Do not change the input frame.

Starter code:

```

1 import pandas as pd
2
3
4 def add_age_group(df):
5     def band(row):
6         if row["age"] < 35:
7             return "16-34"
8         if row["age"] < 55:
9             return "35-54"
10        return "55+"
11
12    df["age_group"] = df.apply(band, axis=1)
13    return df

```

Exercise 6. Spot the bug (level 5)

The docstring promises that missing minutes stay missing. A respondent with no minutes recorded shows as "light". Tap the line with the bug, then fix the function.

```

1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
3     minutes stay missing."""
4     minutes = df["minutes"]
5     band = np.where(minutes >= 60, "heavy", "light")
6     return df.assign(viewing_band=band)

```

Write down the line number of the bug. Then write the fix.

Sources

- Row-wise apply was about 500 times slower than `np.where` on one million rows. (verified: Measured in this repo's build environment: pandas 3.0.6, NumPy 2.4.6, 4.03 s against 7.5 ms.)
- `pd.cut` keeps NaN as missing and supports `right=False`. (verified: Run in pandas 3.0.6.)

Track B best-practice checklist

- ☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.
- ☐ ruff and mypy pass before every commit.
- ☐ Functions are pure where possible. Side effects sit at the edges.
- ☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.
- ☐ Inputs are validated with pandera or pydantic at every boundary.
- ☐ Seeds, config and data versions are recorded for every result.
- ☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.
- ☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.
- ☐ The metric matches the decision. Calibration and subgroup performance are checked.
- ☐ Every experiment is tracked in MLflow.
- ☐ Tests cover edge cases, including empty inputs and zeros.
- ☐ Every model has a model card. Every decision has a log entry.

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C2 Lakehouse, Warehouse and SQL analytics endpoint

Roadmap: Months 16 to 18: MLOps and deployment; Current work. When to use each.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

Concept

Fabric gives you three ways to hold and query tables. All three store data as Delta tables in OneLake. They differ in who writes the data and in which language.

Lakehouse. You write with Spark: notebooks, Spark job definitions, Dataflows Gen2 or pipelines. It holds files and Delta tables side by side. Choose it for raw files, PySpark transformations and data science. This is the closest match to your Databricks work.

SQL analytics endpoint. Every lakehouse gets one automatically. It gives a **read-only** T-SQL surface over the lakehouse's Delta tables. You can `SELECT`, and you can create views, functions and stored procedures. You cannot `INSERT`, `UPDATE` or `DELETE` table data through it. It picks up new or changed tables through a metadata sync. Microsoft documents the usual lag as under one minute, but it varies.

Warehouse. You write with T-SQL. It supports full DML and DDL, and multi-table transactions. Choose it when a team works in T-SQL, loads data with stored procedures, or needs several tables to change together or not at all.

One rule decides most cases: **who writes the data, and in which language?** Spark writers go to a lakehouse. T-SQL writers go to a warehouse. T-SQL readers of lakehouse data use the SQL analytics endpoint. You do not have to copy data between them. A warehouse query reads lakehouse tables in the same workspace with a three-part name: `lakehouse_name.schema.table`. Direct Lake semantic models read from either. Check the status of each feature you rely on in `VERSIONS.md` before you build. Fabric changes monthly.

Why it matters

A team moves a finance reconciliation from SQL Server to Fabric. They land every table in a lakehouse, because the Databricks team already uses lakehouses. The month-end stored procedure updates three tables. On the SQL analytics endpoint it fails on its first `UPDATE`, because the endpoint is read-only.

Under deadline, someone rewrites it as three separate Spark notebook writes. On one month-end the second notebook fails. Table one holds the new month and table three holds the old one, and the reconciliation report shows a balance that does not tie. A warehouse transaction would have rolled back all three tables together.

Worked example: weak code

```
❶ -- Connected to the SQL analytics endpoint of lakehouse lh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
❷    UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
5    INSERT INTO dbo.balances
6    SELECT account_id, @period, SUM(amount) FROM dbo.ledger
7    WHERE period = @period GROUP BY account_id;
❸    DELETE FROM dbo.open_items WHERE period = @period;
9 END;
```

1. **Line 1.** The endpoint gives read-only access to lakehouse tables. Every write below fails.
2. **Line 4.** `UPDATE`, `INSERT` and `DELETE` on lakehouse tables are not supported through the endpoint.
3. **Line 8.** Three tables must change together. Without a transaction, a failure halfway leaves them inconsistent.

Worked example: strong code

```
❶ -- Connected to warehouse wh_finance
2 CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3 BEGIN
❷    BEGIN TRANSACTION;
5    UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
❸    INSERT INTO dbo.balances (account_id, period, balance)
7    SELECT account_id, @period, SUM(amount) FROM dbo.ledger
8    WHERE period = @period GROUP BY account_id;
9    DELETE FROM dbo.open_items WHERE period = @period;
10   COMMIT TRANSACTION;
11 END;
```

1. **Line 1.** The T-SQL writer works in a warehouse, which supports full DML.

2. **Line 4.** All three changes commit together or not at all. Warehouses support multi-table transactions.
3. **Line 6.** Name the target columns. An INSERT without a column list breaks when someone adds a column.

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Recap card

- **Lakehouse, Warehouse or endpoint: the deciding question** Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
- **What can you do on the SQL analytics endpoint?** SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
- **When do you need a Warehouse over a Lakehouse?** T-SQL writes, stored procedure loads, and multi-table transactions.
- **Read a lakehouse table from a warehouse in the same workspace** Use a three-part name: lakehouse.schema.table. No copy.
- **A new lakehouse table is missing from the endpoint** Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

Practice

Exercise 1. Multiple choice (level 1)

Which statement about a lakehouse's SQL analytics endpoint is true?

- a) You can run UPDATE on lakehouse tables through it.
- b) You can create a view over lakehouse tables in it.
- c) You create it by hand after you create the lakehouse.
- d) It stores its own copy of the lakehouse tables.

Why?

- r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.
- r2) Views are stored inside the Delta tables.

Exercise 2. Architecture design (level 2)

Pick the Fabric item for each workload.

Options: Lakehouse, Warehouse, SQL analytics endpoint.

1. Land raw JSON survey deliveries and clean them with PySpark
2. Finance loads budgets with T-SQL stored procedures that change several tables together
3. Analysts query the cleaned lakehouse tables with T-SQL and save a few views

Exercise 3. Spot the bug (level 3)

This script runs against the SQL analytics endpoint of `lh_research`. It fails partway. Tap the failing line, then rewrite the script so it runs on the endpoint.

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W00';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a view `dbo.v_wave_reach` with one row per `wave_id` and a column `reach_pct`: the sum of `weighted_reach` divided by the sum of `weighted_base` from `dbo.nat_metrics`. A wave with a zero base must return NULL, not a divide-by-zero error.

Starter code:

```
1 CREATE VIEW dbo.v_wave_reach AS
```

Exercise 5. Fill the gap (level 4)

You are connected to warehouse `wh_finance`. Lakehouse `lh_research` sits in the same workspace. Complete the query so it reads the lakehouse table without copying it.

```
1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM _____.dbo.nat_metrics
4 GROUP BY wave_id;
```

Exercise 6. Multiple choice (level 5)

A pipeline runs a Spark notebook that creates a new gold table in `lh_research`. The next pipeline step immediately queries the table through

the SQL analytics endpoint and fails with "Invalid object name". An hour later the same query works. What is the most likely cause?

- a) The notebook wrote the table in a format the endpoint cannot read.
- b) The endpoint had not yet synced the new table's metadata.
- c) The endpoint needs a manual refresh every time a table changes.
- d) The pipeline's identity lacks permission on new tables for one hour.

Why?

- r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.
- r2) The endpoint copies data from the lakehouse on a schedule.

Practical: Lakehouse, endpoint and warehouse side by side

Platform: Fabric trial workspace. Run this in a Fabric trial workspace. You create a lakehouse and a warehouse, write one table with Spark, read it through the endpoint, and see a write fail there.

1. In your trial workspace, create a lakehouse named `lh_research`.
2. Open a new notebook attached to `lh_research`. Run:

```
spark.createDataFrame([("W01", 120.0, 400.0), ("W02", 0.0, 0.0)],  
"wave_id STRING, weighted_reach DOUBLE, weighted_base  
DOUBLE").write.mode("overwrite").saveAsTable("nat_metrics").
```
3. Switch the lakehouse view to its SQL analytics endpoint. Wait for `nat_metrics` to appear. Note how long it takes.
4. Open a new SQL query on the endpoint and create the `dbo.v_wave_reach` view from exercise 4. Query it. Wave W02 returns NULL.
5. In the same query window, run `UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W01';` and read the error message.
6. Create a warehouse named `wh_finance`. In a new query, run `SELECT * FROM lh_research.dbo.nat_metrics;`. If it fails, record the error, since this lesson marks cross-item queries as unverified.
7. Write down what you saw in each step. Compare it with the recap card.

Sources

- The SQL analytics endpoint is read-only for Delta table data. No INSERT, UPDATE or DELETE. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint>)

- You can create views, functions and stored procedures in the SQL analytics endpoint. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint>)
- Fabric Warehouse supports full DML, DDL and multi-table transactions. (search-only: <https://learn.microsoft.com/en-us/fabric/fundamentals/decision-guide-lakehouse-warehouse>)
- Metadata sync lag is usually under one minute, and you can refresh it from the portal, the REST API or `sys.sp_dw_refresh_ext_table`. (search-only: <https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-metadata-sync>)
- A warehouse reads lakehouse tables in the same workspace with three-part names. (unverified: Check Microsoft Learn: cross-database queries in Fabric Data Warehouse.)

Track C best-practice checklist

- ☐ Check the status (GA or preview) of every Fabric feature before you depend on it.
- ☐ One copy of each dataset. Use shortcuts or mirroring before you copy.
- ☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.
- ☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.
- ☐ Semantic models use star schemas. DAX measures state their filter context.
- ☐ Direct Lake models are checked for fallback to DirectQuery.
- ☐ Every Copilot output is read, run on known data and compared against a second method.
- ☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.
- ☐ Items live in Git. Changes move through deployment pipelines.
- ☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D3 Silent errors

Roadmap: Every month. Join explosion, nulls in filters, type coercion, timezone drift, mutable defaults.

D3-L3 Mutable defaults and the truthiness of zero

Concept

Two Python habits produce wrong results with no error message. Both appear in code written by people and by AI assistants, including your reference pipeline.

Mutable default arguments. Python evaluates a default value once, when it defines the function. Every call that relies on the default shares that one object. Write `def add_check(result, results=[])`, and each call appends to the same list. The second validation run returns the first run's checks as well.

The fix is a `None` sentinel:

```
def add_check(result: str, results: list[str] | None = None) -> list[str]:
    results = [] if results is None else results
    results.append(result)
    return results
```

ruff flags the pattern as rule B006. Treat that warning as a bug.

Truthiness of numbers. `if value:` is `False` for `None`, and also for `0`, `0.0`, empty strings and empty containers. When zero is a valid measurement, such as a 0% reach, a zero weight or zero rows, a truthiness test treats it as missing.

Your reference `check_wave_stability` does this:

```
if current_pct and prior_pct:
```

A metric that falls from 12% to 0% skips the comparison. The largest possible drop produces no warning.

Test for the case you mean. For missing, write `is None`. For zero, write `== 0`.

```
if current_pct is not None and prior_pct is not None:
```

The same rule covers type hints. `error_message: str = None` claims the value is always a string. Write `str | None = None`, and mypy then makes you handle the `None` case.

Why it matters

A questionnaire routing error removes one question for a whole wave. Its weighted percentage drops from 12% to 0.0%. The wave-over-wave stability check runs `if current_pct and prior_pct:`, treats 0.0 as missing and skips the metric. QA reports PASS. The dashboard publishes 0% reach for that question, and a client asks why the channel "lost its whole audience" in a week.

The check existed to catch exactly this change. One truthiness test disabled it for the one value that mattered most.

Worked example: weak code

```
2 def compare_waves(current: dict, prior: dict, tolerance: float = 0.10,
  flags=[]):
3     """Flag metrics whose share moved more than tolerance between
  waves."""
4     for metric, current_pct in current.items():
5         prior_pct = prior.get(metric)
6         if current_pct and prior_pct:
7             if abs(current_pct - prior_pct) > tolerance:
8                 flags.append(metric)
9     return flags
```

1. **Line 1.** Bare `dict` hides the value type. The values are floats or `None`.
2. **Line 1.** The default list is created once. Flags from one call leak into the next.
3. **Line 5.** A share of 0.0 is falsy, so the biggest possible drop is never compared.

Worked example: strong code

```
1 def compare_waves(
2     1 current: dict[str, float | None],
3     prior: dict[str, float | None],
4     tolerance: float = 0.10,
5 ) -> list[str]:
6     """Flag metrics whose share moved more than tolerance between waves.
7
8     A metric is skipped only when a share is missing. A share of 0.0 is
  compared.
9     """
10    flags: list[str] = []
11    for metric, current_pct in current.items():
12        prior_pct = prior.get(metric)
13        3 if current_pct is None or prior_pct is None:
```

```

14         continue
15     if abs(current_pct - prior_pct) > tolerance:
16         flags.append(metric)
17     return flags

```

1. **Line 2.** The hint says a share can be missing, so mypy checks every use.

2. **Line 10.** A new list on every call. No state leaks between runs.

3. **Line 13.** Skips only missing values. 0.0 goes on to the comparison.

Both fixes are one line. Both bugs pass every test that uses only non-zero values and a single call.

Recap card

- **When does Python evaluate a default argument?** Once, when it defines the function. A list or dict default is shared by every call.
- **Safe pattern for an optional list argument** Default to None. Inside: `results = [] if results is None else results.`
- **What is wrong with `if current_pct and prior_pct:`?** 0.0 is falsy, so a real share of zero is treated as missing. Use `is not None`.
- **Type hint for an optional string that defaults to None** `str | None = None`. Never `str = None`.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```

1 def add_flag(flag, flags=[]):
2     flags.append(flag)
3     return flags
4
5 add_flag("null_weights")
6 print(add_flag("row_count"))

```

Exercise 2. Multiple choice (level 2)

`current_pct = 0.0` and `prior_pct = 0.12`. The tolerance is 0.10. What does `check_wave_stability` from the reference code do with this metric?

- a) Flags it as WARN, because the change is 0.12.
- b) Marks it PASS.
- c) Leaves it out of the results.
- d) Raises a ZeroDivisionError.

Why?

- r1) `if current_pct and prior_pct:` is False when `current_pct` is 0.0, so the block never runs.
- r2) The function filters out metrics below 1%.

Exercise 3. Fill the gap (level 3)

Fix both signatures. The error message is optional, and the flags list must not be shared between calls.

```
1 def log_run(status: str, error_message: ____ (1) = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = ____ (2)) ->
    list[str]: ...
```

Exercise 4. Spot the bug (level 3)

This check should warn when a share moves by more than the tolerance. A drop from 12% to 0% produces no warning. Tap the line with the bug, then fix the function.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct and prior_pct:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `compare_waves(current, prior, tolerance=0.10)`. Both arguments map metric names to a share or `None`. Return the metrics whose share changed by more than `tolerance`, in the order of `current`. Skip a metric only when either share is `None` or the metric is missing from `prior`. Calling the function twice must not carry results over.

Starter code:

```
1 def compare_waves(
2     current: dict[str, float | None],
3     prior: dict[str, float | None],
4     tolerance: float = 0.10,
5 ) -> list[str]:
6     ...
```

Exercise 6. Refactor (level 5)

A validator collects check results with the helper below. The second contract validation in a job reports the first contract's failures too. Refactor `add_result` so each validation starts empty, keep the signature compatible for callers that pass a list, and add type hints and a docstring.

Starter code:

```
1 def add_result(name, passed, results=[]):
2     results.append({"check": name, "passed": passed})
3     return results
```

Sources

- ruff rule B006 flags mutable default arguments. (verified: ruff 0.16.9 run in this repo reports B006 on `def f(x=[])`.)

Track D best-practice checklist

- ☐ Read a traceback from the bottom up. Find the first frame in your own code.
- ☐ Reproduce the error on the smallest data that shows it.
- ☐ Check row counts before and after every join.
- ☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.
- ☐ Never use a mutable object as a default argument.
- ☐ Check every function and parameter an AI suggests against the official docs for your version.
- ☐ Run AI code on a small input with a known answer before you trust it.
- ☐ Write the failing test first. A test that only repeats the code proves nothing.
- ☐ Ask the model to list its assumptions. Check each one.
- ☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A3-L1 Idempotent MERGE, and why you deduplicate the source first

A3-L1 exercise 1

Hints. (1) Count how many source rows match the one target row. (2) Ask yourself which of the two source rows Delta should use for the update. (3) Delta raises `DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE` in this case.

Answer. b) The MERGE fails with an error. Because: r1) Two source rows match one target row, and Delta cannot tell which should win.

Delta requires at most one source row per matched target row. With two, the update is ambiguous, so the `MERGE` fails with

`DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE`. Deduplicate the source first.

Common wrong answers.

- a: Delta has no notion of 'last' in an unordered DataFrame. Row order in Spark is not guaranteed, so Delta refuses to guess.
- d: Spark gives no row order without an explicit sort, so 'first' has no meaning. Delta refuses to guess.
- c: Both rows match the existing target row, so neither goes to the insert clause. Duplicate inserts happen only when no target row matches.
- b|r2: Right outcome, wrong reason. Delta does not enforce primary keys. The error comes from the ambiguous match, not from a key constraint.
- b|r3: Right outcome, wrong reason. Caching has nothing to do with it. The error comes from two source rows matching one target row.

A3-L1 exercise 2

Hints. (1) Two clauses and one action. (2) The clause names start with `whenMatched` and `whenNotMatched`, and both copy every column. (3) The copy-every-column versions end in `All`. The action that runs a MERGE is not `collect`.

Answer.

```
1 (
2   DeltaTable.forName(spark,
3     "research_silver.nat_respondents").alias("t")
4   .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
5     t.wave_id = s.wave_id")
6   .whenMatchedUpdateAll()
7   .whenNotMatchedInsertAll()
8   .execute()
9 )
```

`whenMatchedUpdateAll()` copies every source column onto the matched target row. `whenNotMatchedInsertAll()` inserts rows with no match. `execute()` runs the MERGE.

Common wrong answers.

- `g3 = collect: collect()` is a DataFrame action. A MERGE builder runs with `execute()`.
- `g1 = whenMatchedUpdate: whenMatchedUpdate()` needs a `set=` mapping of columns. With no arguments you want `whenMatchedUpdateAll()`.
- `g2 = whenNotMatchedBySourceDelete: whenNotMatchedBySourceDelete()` deletes target rows that are missing from the source. With a one-wave source it would delete every other wave.

A3-L1 exercise 3

Hints. (1) Work through the first run: which clause fires for each row? (2) On the second run, does any source row lack a match? (3) Matched rows update in place. Updates do not add rows.

Answer.

```
1 3
```

Run one inserts 3 rows. Run two matches all 3 and updates them with identical values. The count stays 3. That is idempotence.

Common wrong answers.

- `6`: 6 is what an append would give. On the second run every source row matches an existing target row, so the MERGE updates instead of inserting.
- `0`: The first run inserts all 3 rows, because none of them matches an empty table.

A3-L1 exercise 4

Hints. (1) The grain is respondent per wave. Which columns identify one row? (2) Compare the deduplication keys with the MERGE condition. (3) A W2 row for respondent R1 matches the W1 row for R1, and overwrites it.

Answer. The bug is on line 6.

Fixed version:

```
1 def upsert_respondents(spark: SparkSession, source: DataFrame) -> None:
2     """MERGE one wave of respondents into silver. Grain: respondent per
   wave."""
3     latest = latest_per_key(source, ["respondent_id", "wave_id"],
   "delivery_version")
4     (
5         DeltaTable.forName(spark,
6         "research_silver.nat_respondents").alias("t")
7         .merge(latest.alias("s"), "t.respondent_id = s.respondent_id AND
8         t.wave_id = s.wave_id")
9         .whenMatchedUpdateAll()
10        .whenNotMatchedInsertAll()
11        .execute()
12    )
```

The condition matches on `respondent_id` alone. A wave W2 row for a respondent matches that respondent's W1 row and overwrites it, including `wave_id`. Match on the full grain: `t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id`.

Common wrong answers.

- Line 3: Line 3 deduplicates on both key columns, which matches the grain. Look at what the MERGE matches on.
- Line 7: `whenMatchedUpdateAll()` is right for a redelivery. The problem is which rows count as a match.

A3-L1 exercise 5

Hints. (1) You need a window, a rank and a filter. (2)

`Window.partitionBy(*keys).orderBy(F.col(order_col).desc())` sorts each key's rows newest first. (3) Add `F.row_number().over(window)` as a column, keep rank 1, then drop the helper column.

Answer.

```
1 from pyspark.sql import DataFrame, Window
2 from pyspark.sql import functions as F
3
4
5 def latest_per_key(df: DataFrame, keys: list[str], order_col: str) ->
   DataFrame:
```

```

6     """Keep the row with the highest order_col for each key.
7
8     Args:
9         df: Source rows, possibly with repeated keys.
10        keys: Columns that identify one target row.
11        order_col: Column that ranks versions. Highest wins.
12
13    Returns:
14        One row per key.
15    """
16    window = Window.partitionBy(*keys).orderBy(F.col(order_col).desc())
17    return (
18        df.withColumn("_rank", F.row_number().over(window))
19        .filter(F.col("_rank") == 1)
20        .drop("_rank")
21    )

```

A window partitioned by the keys and ordered by `order_col` descending puts the newest row first in each key. `row_number()` numbers them 1, 2, 3. Keep 1, drop the helper column.

A3-L1 exercise 6

Hints. (1) Nothing unchecked should reach silver. (2) The MERGE needs a source with one row per key. (3) The run log records the outcome of everything before it.

Answer.

1. Validate the bronze delivery against its data contract
2. Rename and cast columns from the Variable Registry
3. Keep the newest row per respondent and wave
4. MERGE into silver on `respondent_id` and `wave_id`
5. Run the table health check on silver
6. Write one record to the pipeline run log

Check the delivery, map it to canonical names, deduplicate, merge, confirm the table is healthy, then log the run with its final status.

Common wrong answers.

- Deduplicating after the MERGE is too late. The MERGE has already failed, or has already inserted the repeated rows.
- The contract describes the delivery as received. Check it before you rename, or the column names in the contract no longer match.
- Log last, so the run record includes the health check result and the final status.

B3-L2 Vectorise instead of apply

B3-L2 exercise 1

Hints. (1) Look for the option with no per-row Python code. (2) A vectorised call takes whole columns as arguments. (3) `np.where(cond, a, b)` picks from `a` or `b` element by element in compiled code.

Answer. c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
Because: r1) It evaluates the condition and the choice on whole arrays in compiled code.

`np.where` takes whole arrays and runs one compiled loop. The other three options call Python code once per row.

Common wrong answers.

- a: `apply(axis=1)` calls the lambda once per row in Python. It measured about 500 times slower than `np.where` on one million rows.
- b: A list comprehension beats `apply`, but it still runs a Python loop over every value.
- d: `iterrows()` builds a Series for each row. It is the slowest option here.
- c|r2: Right choice, wrong reason. Length has nothing to do with speed. `np.where` works on whole arrays in compiled code.
- c|r3: Right choice, wrong reason. There is no lambda, and `np.where` runs on one core. It is fast because it avoids a Python call per row.

B3-L2 exercise 2

Hints. (1) `None` in a float Series becomes `NaN`. (2) What does any comparison with `NaN` return? (3) Every comparison with `NaN` is `False`.

Answer.

```
1 [False, True, False]
```

`None` becomes `NaN` in a float64 Series, and `NaN > 1` is `False`. So the missing value is treated as "not above 1". This is the missing-value trap from the concept.

Common wrong answers.

- `[False, True, None]`: A comparison on a float Series returns plain booleans. `NaN > 1` evaluates to `False`, not to a missing value.
- `[False, True, nan]`: Comparisons do not propagate `NaN` on float64 data. They return `False`, which is why missing values get silently mislabelled.
- `[True, True, False]`: `0.0 > 1` is `False`. Only 1.5 exceeds 1.

B3-L2 exercise 3

Hints. (1) The function takes a condition, a value for True and a value for False. (2) Its name is the SQL keyword for filtering rows. (3) `np.where(cond, value_if_true, value_if_false)`, with zero as the false value.

Answer.

```
1 df["weighted_reach"] = np.where(df["reach_flag"] == 1,
    df["final_weight"], 0.0)
```

`np.where(cond, df["final_weight"], 0.0)` keeps the weight where the flag is 1 and returns 0.0 elsewhere, as one float array.

Common wrong answers.

- `g1 = select`: `np.select` takes a list of conditions and a list of choices. For one condition and two outcomes, use `np.where`.
- `g1 = if`: There is no `np.if`. The vectorised if-else in NumPy is `np.where`.
- `g2 = None`: `None` turns the result into an object array. Use `0.0` so the column stays numeric.

B3-L2 exercise 4

Hints. (1) Compute the total weight first, and return early when it is zero. (2) `df['final_weight'].where(df['reach_flag'] == 1, 0.0)` keeps weights for reached rows only. (3) Sum the masked weights, divide by the total, and wrap the result in `float()`.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     """Weighted share of respondents with reach_flag == 1.
7
8     Args:
9         df: Rows with reach_flag and final_weight.
10
11     Returns:
12         The weighted share, or None when the total weight is zero.
13     """
14     total = df["final_weight"].sum()
15     if total == 0:
16         return None
17     reached = df["final_weight"].where(df["reach_flag"] == 1, 0.0).sum()
18     return float(reached / total)
```

Sum the weights once for the denominator and return `None` when it is zero, which covers the empty frame too. Mask weights by the flag, sum, divide and convert to `float`.

B3-L2 exercise 5

Hints. (1) `pd.cut` turns numbers into labelled bands and keeps missing values missing. (2) Bins `[-np.inf, 35, 55, np.inf]` with `right=False` give `[.., 35)`, `[35, 55)`, `[55, ..)`. (3) Wrap it in `df.assign(age_group=pd.cut(...))` so you return a new frame.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4 AGE_BINS = [-np.inf, 35, 55, np.inf]
5 AGE_LABELS = ["16-34", "35-54", "55+"]
6
7
8 def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
9     """Return a copy of df with an age_group column. Missing ages stay
10     missing."""
11     return df.assign(
12         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
13             right=False)
14     )
```

`pd.cut` with left-closed bins (`right=False`) matches the original `<` comparisons exactly. It keeps `NaN` as missing, which fixes a bug the original had. `assign` returns a new frame.

B3-L2 exercise 6

Hints. (1) What is `NaN >= 60`? (2) `np.where` has only two outcomes, so a missing value gets one of them. (3) Build the band as a Series, then keep it only where `minutes.notna()`.

Answer. The bug is on line 4.

Fixed version:

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
3     minutes stay missing."""
4     minutes = df["minutes"]
5     band = pd.Series(np.where(minutes >= 60, "heavy", "light"),
6         index=df.index)
7     return df.assign(viewing_band=band.where(minutes.notna()))
```

`NaN >= 60` is `False`, so `np.where` returns `"light"` for missing minutes. Wrap the result in a Series on the same index and apply `.where(minutes.notna())` to restore the missing values.

Common wrong answers.

- Line 3: Line 3 only selects the column. The missing value is lost where the condition is evaluated.
- Line 5: `assign` is right. It writes whatever `band` holds, and `band` is already wrong.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

C2-L3 exercise 1

Hints. (1) The endpoint is described as read-only. Read-only for what? (2) It blocks changes to table data, not every kind of object. (3) Views, functions and stored procedures that read are allowed.

Answer. b) You can create a view over lakehouse tables in it. Because: r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

The endpoint is read-only for table data, but you can create views, functions and stored procedures in it. Fabric creates it with the lakehouse, and it reads the same Delta tables in OneLake.

Common wrong answers.

- a: The endpoint is read-only for table data. Write with Spark in the lakehouse, or move the table to a warehouse.
- c: Fabric creates the endpoint automatically with every lakehouse.
- d: It reads the same Delta tables in OneLake. There is no second copy. It syncs metadata, not data.
- b | r2: Right choice, wrong reason. A view is a T-SQL object in the endpoint. The Delta tables stay unchanged.

C2-L3 exercise 2

Hints. (1) Ask who writes the data, and in which language. (2) Spark writers and files: one item. T-SQL writers: another. (3) T-SQL readers of lakehouse data need no new item at all.

Answer.

- Raw files and PySpark: **Lakehouse**. Spark writes, and files sit next to tables.
- T-SQL writes across several tables: **Warehouse**, for DML and multi-table transactions.
- T-SQL reads of lakehouse data, plus views: **SQL analytics endpoint**. No copy needed.

Spark writers use a lakehouse. T-SQL writers use a warehouse. T-SQL readers of lakehouse data use the endpoint that comes with the lakehouse.

Common wrong answers.

- finance = lakehouse: The finance team writes with T-SQL. A lakehouse accepts writes from Spark, and its endpoint is read-only. Use a warehouse.
- finance = endpoint: The endpoint cannot change table data, so the stored procedures would fail on their first UPDATE or INSERT.
- raw = warehouse: A warehouse does not hold raw files, and the team writes with PySpark. Use a lakehouse.
- adhoc = warehouse: Copying the lakehouse tables into a warehouse makes a second copy to keep in sync. The endpoint reads them where they are.

C2-L3 exercise 3

Hints. (1) Which statements change table data? (2) The endpoint accepts reads and objects that read. (3) UPDATE on a lakehouse table fails through the endpoint.

Answer. The bug is on line 7.

Fixed version:

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 -- Fix wave W00 in the lakehouse with Spark, not here.
```

Line 7 is an UPDATE on a lakehouse table, which the read-only endpoint rejects. Make that correction in a Spark notebook against the lakehouse, and keep only the view here.

Common wrong answers.

- Line 2: Creating a view is allowed on the endpoint. Look for a statement that changes table data.

- Line 3: The division could hit zero, but that is not what stops this script. Look for a write.

C2-L3 exercise 4

Hints. (1) Aggregate both sums per wave, then divide. (2) `NULLIF(x, 0)` returns NULL when x is 0, and dividing by NULL gives NULL. (3)

`SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct`, then `GROUP BY wave_id`.

Answer.

```
1 CREATE VIEW dbo.v_wave_reach AS
2 SELECT
3     wave_id,
4     SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
5 FROM dbo.nat_metrics
6 GROUP BY wave_id;
```

Sum both columns per wave and divide by `NULLIF(SUM(weighted_base), 0)`. A zero base becomes NULL, so the division returns NULL instead of an error.

C2-L3 exercise 5

Hints. (1) Three-part names are `item.schema.table`. (2) The first part names the item that holds the table. (3) The table sits in the lakehouse `lh_research`.

Answer.

```
1 INSERT INTO dbo.wave_summary (wave_id, weighted_base)
2 SELECT wave_id, SUM(weighted_base)
3 FROM lh_research.dbo.nat_metrics
4 GROUP BY wave_id;
```

A warehouse query reads a lakehouse table in the same workspace with `lh_research.dbo.nat_metrics`. The data stays in one place.

Common wrong answers.

- `g1 = wh_finance: wh_finance` is the warehouse you are in. The table lives in the lakehouse, so name the lakehouse.
- `g1 = OneLake`: There is no database called OneLake. The first part of a three-part name is the item name: the lakehouse.

C2-L3 exercise 6

Hints. (1) The query failed first and worked later, with no change in between. (2) How does the endpoint learn that a new table exists? (3) Look at the metadata sync between the lakehouse and its endpoint.

Answer. b) The endpoint had not yet synced the new table's metadata. Because: r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.

The endpoint syncs lakehouse metadata automatically, usually in under a minute, but the lag varies. When a later step depends on a new table, add a metadata refresh step (portal, REST API) or a retry before the query. Check the current refresh options on Microsoft Learn.

Common wrong answers.

- a: The query worked an hour later without any change to the table, so the format was fine.
- c: The sync runs automatically. A manual refresh speeds it up, but it is not required every time.
- d: Permissions do not change on a timer. The delay points to the metadata sync.
- b|r2: Right choice, wrong reason. The endpoint does not copy data. It syncs metadata about the Delta tables.

D3-L3 Mutable defaults and the truthiness of zero

D3-L3 exercise 1

Hints. (1) When is the default `[]` created? (2) Both calls use the default. Is it the same list? (3) The list is created once, when Python defines the function.

Answer.

```
1 ['null_weights', 'row_count']
```

The default list is created when Python defines the function. Both calls append to that one list, so the second call returns both flags.

Common wrong answers.

- `['row_count']`: That is what the author intended. The default list is created once, so the second call appends to the list the first call used.
- `['row_count', 'null_weights']`: `append` adds to the end. The first call's flag is already in the list.

D3-L3 exercise 2

Hints. (1) Evaluate `0.0` and `0.12` in your head. (2) `0.0` is falsy, like `None`. (3) The whole block, including `results.append`, is skipped.

Answer. c) Leaves it out of the results. Because: `r1) if current_pct and prior_pct:` is False when `current_pct` is 0.0, so the block never runs.

`0.0 and 0.12` evaluates to `0.0`, which is falsy. The block that compares and appends never runs, so the metric vanishes from the results and no warning appears.

Common wrong answers.

- a: That is what the check should do. Look at the `if` line before the comparison: `0.0` is falsy.
- b: Nothing is appended for this metric, so it cannot be marked PASS. The `if` skips it.
- d: The check subtracts, it does not divide. No error appears, which is the problem.
- c|r2: Right outcome, wrong reason. There is no 1% filter. The truthiness test treats `0.0` as missing.

D3-L3 exercise 3

Hints. (1) The first default is `None`, so the type must allow `None`. (2) The second default must be immutable. (3) `str | None` for the first gap and `None` for the second.

Answer.

```
1 def log_run(status: str, error_message: str | None = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = None) -> list[str]: ...
```

`error_message: str | None = None` states that the value may be missing.

`flags: list[str] | None = None` avoids a shared list. Create the list inside the function when it is `None`.

Common wrong answers.

- `g1 = str: str = None` claims the value is always a string. mypy in strict mode rejects it. Write `str | None`.
- `g2 = []`: A list default is created once and shared by every call. Use `None` and create the list inside the function.

D3-L3 exercise 4

Hints. (1) Which values make `if current_pct and prior_pct:` False? (2) `0.0` is one of them. (3) Replace the truthiness test with `is not None` on both values.

Answer. The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
   tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct is not None and prior_pct is not None:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Line 3 treats 0.0 as missing. Use `if current_pct is not None and prior_pct is not None:` so that only missing shares skip the comparison.

Common wrong answers.

- Line 4: The subtraction is correct. The code never reaches it when a share is 0.0.
- Line 5: The comparison with the tolerance is correct. Look at what stops the code reaching it.

D3-L3 exercise 5

Hints. (1) Start with `flags: list[str] = []` inside the function. (2) Read the prior share with `prior.get(metric)`, which gives `None` for a missing metric. (3) Skip with `if current_pct is None or prior_pct is None: continue`, then compare with the tolerance.

Answer.

```
1 def compare_waves(
2     current: dict[str, float | None],
3     prior: dict[str, float | None],
4     tolerance: float = 0.10,
5 ) -> list[str]:
6     """Flag metrics whose share moved more than tolerance between waves.
7
8     A metric is skipped only when a share is missing. A share of 0.0 is
9     compared.
10    """
11     flags: list[str] = []
12     for metric, current_pct in current.items():
13         prior_pct = prior.get(metric)
14         if current_pct is None or prior_pct is None:
15             continue
16         if abs(current_pct - prior_pct) > tolerance:
17             flags.append(metric)
18     return flags
```

Create the list inside the function. Use `prior.get` so a new metric gives `None`. Skip only on `None`, then compare the absolute change with the tolerance.

D3-L3 exercise 6

Hints. (1) Default `results` to `None`. (2) Create a new list only when `results` is `None`. (3) `results or []` looks similar but replaces an empty list the caller passed. Use `is None`.

Answer.

```
1 def add_result(  
2     name: str, passed: bool, results: list[dict[str, object]] | None =  
3     None  
4 ) -> list[dict[str, object]]:  
5     """Append one check result and return the list.  
6  
7     Args:  
8         name: Check name.  
9         passed: Whether the check passed.  
10        results: List to append to. A new list is created when omitted.  
11  
12    Returns:  
13        The list with the new result.  
14    """  
15    results = [] if results is None else results  
16    results.append({"check": name, "passed": passed})  
17    return results
```

Default to `None`, create the list when `results` is `None`, then append and return. Avoid `results or []`: an empty list the caller passed is falsy, so the function would append to a new list and the caller would never see the result.